СОГЛАСОВАНО

Председатель муниципальной предметно-методической комиссии
по информатике

_____ /_____/
«___» _____ 2026 г.

УТВЕРЖДАЮ

Руководитель органа местного самоуправления
в сфере образования

_____ /_____/
«___» _____ 2026 г.

Муниципальный этап ВсОШ по информатике

Решения задач (7-8 классы)

Учебный год: **2026/27**

Задача A. Шторы-мини

Условие (кратко)

Дан массив высот:

`3 3 1 4 4 0 2 2 5 1`

Операция: выбрать отрезок и уменьшить на 1 все положительные элементы отрезка.

Требуется найти минимум операций для зануления массива.

Идея

Минимальное число операций равно:

`h1 + sum(max(0, hi - h(i-1)))`

где `h1..hn` — массив высот.

Вычисление

`3 + 0 + 0 + 3 + 0 + 0 + 2 + 0 + 3 + 0 = 11`

Ответ

`11`

Задача B. Лифт-инвертор

Условие (кратко)

Этажи `1..7`, старт `X=3`, команды:

`UUUSDDDDSSUUUD`

`S` меняет смысл `U` и `D`. Выход за границы считается ударом, лифт остается на месте.

Идея

Нужна прямая симуляция:
1. хранить текущий этаж;

2. хранить флаг инверсии;
3. считать удары о границу.

Результат симуляции

- конечный этаж: `5`;
- ударов о границу: `3`.

Ответ

`5 3`

Задача C. Маятниковая строка

Условие (кратко)

Для строки `s`:
- символы на нечетных позициях добавляются в конец;
- символы на четных — в начало.

Идея решения

Использовать двустороннюю очередь (`deque`):
- если позиция `i` нечетная -> `push_back(s[i])`;
- если четная -> `push_front(s[i])`.

После обработки всех символов выводим содержимое `deque`.

Корректность

Операции `push_front/push_back` точно повторяют требование задачи для каждой позиции, поэтому итоговая строка совпадает с определением в условии.

Сложность

- Время: `O(n)`
- Память: `O(n)`

Эталон (Python)

```
from collections import deque

n = int(input().strip())
s = input().strip()

dq = deque()
for i, ch in enumerate(s, start=1):
    if i % 2 == 1:
        dq.append(ch)
    else:
        dq.appendleft(ch)

print(''.join(dq))
```

Задача D. Шторы-online

Условие (кратко)

Для произвольного массива `h1..hn` нужно найти минимум операций той же модели, что в задаче A.

Идея решения

Используем ту же формулу:

`ans = h1 + sum(max(0, hi - h(i-1)))`

Почему это работает:
- если текущая высота выше предыдущей, нужно запустить еще `hi - h(i-1)` новых «слоев» операций;
- спад высоты дополнительной операции не требует.

Сложность
- Время: `O(n)`
- Память: `O(1)` дополнительная

Эталон (Python)

```
n = int(input().strip())
h = list(map(int, input().split()))

ans = h[0]
for i in range(1, n):
    if h[i] > h[i - 1]:
        ans += h[i] - h[i - 1]

print(ans)
```

Задача E. Дублирующий редактор

Условие (кратко)
Нужно получить строку `s` из пустой строки:
1. `ADD` — добавить следующий символ `s`;
2. `DUP` — удвоить текущую строку.

Найти минимальное число операций.

Идея решения
Динамика по длине префикса:
- `dp[i]` — минимум операций для получения первых `i` символов.

Переходы:
1. `ADD`: `dp[i+1] = min(dp[i+1], dp[i] + 1)`.
2. `DUP`: если `2*i <= n` и `s[0:i] == s[i:2*i]`, тогда `dp[2*i] = min(dp[2*i], dp[i] + 1)`.

Проверку равенства блоков делаем за `O(1)` после линейной предобработки `Z`-функцией:
- условие равенства для длины `i`: `z[i] >= i`.

Корректность
Любая допустимая стратегия строит строку как последовательность состояний по длине.
Переходы в `dp` перечисляют все допустимые типы шага (`ADD`/`DUP`) и берут минимум, значит `dp[n]` — оптимален.

Сложность
- Время: `O(n)`
- Память: `O(n)`

Эталон (Python)

```
INF = 10**18
s = input().strip()
```

```
n = len(s)

# Z-функция
z = [0] * n
l = r = 0
for i in range(1, n):
    if i <= r:
        z[i] = min(r - i + 1, z[i - l])
    while i + z[i] < n and s[z[i]] == s[i + z[i]]:
        z[i] += 1
    if i + z[i] - 1 > r:
        l, r = i, i + z[i] - 1

dp = [INF] * (n + 1)
dp[0] = 0

for i in range(n + 1):
    if dp[i] == INF:
        continue
    if i < n:
        dp[i + 1] = min(dp[i + 1], dp[i] + 1)
    if i > 0 and 2 * i <= n and z[i] >= i:
        dp[2 * i] = min(dp[2 * i], dp[i] + 1)

print(dp[n])
```